

CSE 3204
Formal Language, Automata and Computability



Lecture 10
Context Free Grammars

Context Free Grammars

- A palindrome is a string that reads the same forward and backward
- string w is a palindrome if and only if $w = w^R$
- 0110 and 11011
- Language of palindromes on $\{0,1\}$ is not a regular language (read by yourself).
- There is a natural recursive definition of when a string of 0s and 1s is in L_{pal}

BASIS: ϵ , 0, and 1 are palindromes.

INDUCTION: If w is a palindrome, so are $0w0$ and $1w1$. No string is a palindrome of 0's and 1's, unless it follows from this basis and induction rule.

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

Figure 5.1: A context-free grammar for palindromes

Definition of Context Free Grammars

There are four important components in a grammatical description of a language:

1. There is a finite set of symbols that form the strings of the language being defined. This set was $\{0, 1\}$ in the palindrome example we just saw. We call this alphabet the *terminals*, or *terminal symbols*.
2. There is a finite set of *variables*, also called sometimes *nonterminals* or *syntactic categories*. Each variable represents a language; i.e., a set of strings. In our example above, there was only one variable, P , which we used to represent the class of palindromes over alphabet $\{0, 1\}$.
3. One of the variables represents the language being defined; it is called the *start symbol*. Other variables represent auxiliary classes of strings that are used to help define the language of the start symbol. In our example, P , the only variable, is the start symbol.
4. There is a finite set of *productions* or *rules* that represent the recursive definition of a language. Each production consists of:
 - (a) A variable that is being (partially) defined by the production. This variable is often called the *head* of the production.
 - (b) The production symbol $\rightarrow$.
 - (c) A string of zero or more terminals and variables. This string, called the *body* of the production, represents one way to form strings in the language of the variable of the head. In so doing, we leave terminals unchanged and substitute for each variable of the body any string that is known to be in the language of that variable.

We saw an example of productions in Fig. 5.1.

We shall represent a CFG G by its four components that is $G = (V, T, P, S)$ where V is the set of variables, T the terminals, P the set of productions and S the start symbol.

Examples

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

CGF representing expression of typical programming language.

$G = (\{E, I\}, T, P, E)$ where T is the set of symbols $\{a, b, 0, 1, +, *, (,)\}$, P is the set of productions.

Figure 5.1: A context-free grammar for palindromes

$$G_{pal} = (\{P\}, \{0, 1\}, A, P)$$

I represents identifier, a language of regular expressions
 $(a+b)(a+b+0+1)^*$

vertical bars. That is, the productions $A \rightarrow \alpha_1, A \rightarrow \alpha_2, \dots, A \rightarrow \alpha_n$ can be replaced by the notation $A \rightarrow \alpha_1 | \alpha_2 | \dots | \alpha_n$. For instance, the grammar for palindromes from Fig. 5.1 can be written as $P \rightarrow \epsilon | 0 | 1 | 0P0 | 1P1$.

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

Derivation Using a Grammar

- Apply the productions of a CFG to infer that certain strings are in the language of a certain variable.
- Recursive inference:

we take strings known to be in the language of each of the variables of the body concatenate them in the proper order with any terminals appearing in the body and infer that the resulting string is in the language of the variable in the head.

- Derivation:

we use the productions from head to body. We expand the start symbol using one of its productions. We further expand the resulting string by replacing one of the variables by the body of one of its productions and so on until we derive a string consisting entirely of terminals.

Derivation Using a Grammar

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$
5. $I \rightarrow a$
6. $I \rightarrow b$
7. $I \rightarrow Ia$
8. $I \rightarrow Ib$
9. $I \rightarrow I0$
10. $I \rightarrow I1$

	String Inferred	For lang- uage of	Production used	String(s) used
(i)	a	I	5	—
(ii)	b	I	6	—
(iii)	$b0$	I	9	(ii)
(iv)	$b00$	I	9	(iii)
(v)	a	E	1	(i)
(vi)	$b00$	E	1	(iv)
(vii)	$a + b00$	E	2	(v), (vi)
(viii)	$(a + b00)$	E	4	(vii)
(ix)	$a * (a + b00)$	E	3	(v), (viii)

Figure 5.3: Inferring strings using the grammar of Fig. 5.2

Derivation Using a Grammar

- $G = \{V, T, P, S\}$ is a CFG. Let $\alpha A \beta$ = a string of terminals and variables with A as a variable.
- α, β are strings in $(V \cup T)^*$ and A in V .

$$\overset{\text{-----}}{A} \rightarrow \gamma$$

- Then $\alpha A \beta \xRightarrow[G]{*} \alpha \gamma \beta$.

BASIS: For any string α of terminals and variables, we say $\alpha \xRightarrow[G]{*} \alpha$. That is, any string derives itself.

INDUCTION: If $\alpha \xRightarrow[G]{*} \beta$ and $\beta \Rightarrow \gamma$, then $\alpha \xRightarrow[G]{*} \gamma$. That is, if α can become β by zero or more steps, and one more step takes β to γ , then α can become γ . Put another way, $\alpha \xRightarrow[G]{*} \beta$ means that there is a sequence of strings $\gamma_1, \gamma_2, \dots, \gamma_n$, for some $n \geq 1$, such that

1. $\alpha = \gamma_1$,
2. $\beta = \gamma_n$, and
3. For $i = 1, 2, \dots, n - 1$, we have $\gamma_i \Rightarrow \gamma_{i+1}$.

If grammar G is understood, then we use $\xRightarrow{*}$ in place of $\xRightarrow[G]{*}$.

Derivation Using a Grammar

$$E \Rightarrow E * E \Rightarrow I * E \Rightarrow a * E \Rightarrow$$

$$a * (E) \Rightarrow a * (E + E) \Rightarrow a * (I + E) \Rightarrow a * (a + E) \Rightarrow$$

$$a * (a + I) \Rightarrow a * (a + I0) \Rightarrow a * (a + I00) \Rightarrow a * (a + b00)$$

The two viewpoints — recursive inference and derivation — are equivalent. That is, a string of terminals w is inferred to be in the language of some variable A if and only if $A \stackrel{*}{\Rightarrow} w$. However, the proof of this fact requires some work.

I **I** **H** **A** **P** **A**

1. $E \rightarrow I$
2. $E \rightarrow E + E$
3. $E \rightarrow E * E$
4. $E \rightarrow (E)$

$$\begin{array}{lll} 5. & I & \rightarrow a \\ 6. & I & \rightarrow b \\ 7. & I & \rightarrow Ia \\ 8. & I & \rightarrow Ib \\ 9. & I & \rightarrow I0 \\ 10. & I & \rightarrow I1 \end{array}$$

Leftmost Derivation

- To restrict the number of choices in deriving a string, it is often useful to require that at each step we replace the leftmost variable by one of its production bodies.
- Denoted by $\Rightarrow_{lm}$ and $\xRightarrow{*}_{lm}$,

$$E \Rightarrow_{lm} E * E \Rightarrow_{lm} I * E \Rightarrow_{lm} a * E \Rightarrow_{lm}$$

$$a * (E) \Rightarrow_{lm} a * (E + E) \Rightarrow_{lm} a * (I + E) \Rightarrow_{lm} a * (a + E) \Rightarrow_{lm}$$

$$a * (a + I) \Rightarrow_{lm} a * (a + I0) \Rightarrow_{lm} a * (a + I00) \Rightarrow_{lm} a * (a + b00)$$

We can also summarize the leftmost derivation by saying $E \xRightarrow{*}_{lm} a * (a + b00)$, or express several steps of the derivation by expressions such as $E * E \xRightarrow{*}_{lm} a * (E)$.

Rightmost Derivation

$$E \Rightarrow_{rm} E * E \Rightarrow_{rm} E * (E) \Rightarrow_{rm} E * (E + E) \Rightarrow_{rm}$$

$$E * (E + I) \Rightarrow_{rm} E * (E + I0) \Rightarrow_{rm} E * (E + I00) \Rightarrow_{rm} E * (E + b00) \Rightarrow_{rm}$$

$$E * (I + b00) \Rightarrow_{rm} E * (a + b00) \Rightarrow_{rm} I * (a + b00) \Rightarrow_{rm} a * (a + b00)$$

This derivation allows us to conclude $E \xRightarrow{*}_{rm} a * (a + b00)$. $\square$

Any derivation has an equivalent leftmost and an equivalent rightmost derivation. That is, if w is a terminal string, and A a variable, then $A \xRightarrow{*} w$ if and only if $A \xRightarrow{*}_{lm} w$, and $A \xRightarrow{*}_{lm} w$ if and only if $A \xRightarrow{*}_{rm} w$. We shall also prove these

Notation for CFG Derivations

There are a number of conventions in common use that help us remember the role of the symbols we use when discussing CFG's. Here are the conventions we shall use:

1. Lower-case letters near the beginning of the alphabet, a , b , and so on, are terminal symbols. We shall also assume that digits and other characters such as $+$ or parentheses are terminals.
2. Upper-case letters near the beginning of the alphabet, A , B , and so on, are variables.
3. Lower-case letters near the end of the alphabet, such as w or z , are strings of terminals. This convention reminds us that the terminals are analogous to the input symbols of an automaton.
4. Upper-case letters near the end of the alphabet, such as X or Y , are either terminals or variables.
5. Lower-case Greek letters, such as α and β , are strings consisting of terminals and/or variables.

There is no special notation for strings that consist of variables only, since this concept plays no important role. However, a string named α or another Greek letter might happen to have only variables.

Language of a Grammar

If $G = (V, T, P, S)$ is a CFG, the *language* of G , denoted $L(G)$, is the set of terminal strings that have derivations from the start symbol. That is,

$$L(G) = \{w \text{ in } T^* \mid S \xRightarrow[G]{*} w\}$$

- If a language L is the language of some CFG then L is a context free language or CFL.

Theorem 5.7: $L(G_{pal})$, where G_{pal} is the grammar of Example 5.1, is the set of palindromes over $\{0, 1\}$.

PROOF: We shall prove that a string w in $\{0, 1\}^*$ is in $L(G_{pal})$ if and only if it is a palindrome; i.e., $w = w^R$.

(If) Suppose w is a palindrome. We show by induction on $|w|$ that w is in $L(G_{pal})$.

BASIS: We use lengths 0 and 1 as the basis. If $|w| = 0$ or $|w| = 1$, then w is ϵ , 0, or 1. Since there are productions $P \rightarrow \epsilon$, $P \rightarrow 0$, and $P \rightarrow 1$, we conclude that $P \xRightarrow{*} w$ in any of these basis cases.

INDUCTION: Suppose $|w| \geq 2$. Since $w = w^R$, w must begin and end with the same symbol. That is, $w = 0x0$ or $w = 1x1$. Moreover, x must be a palindrome; that is, $x = x^R$. Note that we need the fact that $|w| \geq 2$ to infer that there are two distinct 0's or 1's, at either end of w .

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

Figure 5.1: A context-free grammar for palindromes

Language of a Grammar

If $w = 0x0$, then we invoke the inductive hypothesis to claim that $P \xRightarrow{*} x$. Then there is a derivation of w from P , namely $P \Rightarrow 0P0 \xRightarrow{*} 0x0 = w$. If $w = 1x1$, the argument is the same, but we use the production $P \rightarrow 1P1$ at the first step. In either case, we conclude that w is in $L(G_{pal})$ and complete the proof.

(Only-if) Now, we assume that w is in $L(G_{pal})$; that is, $P \xRightarrow{*} w$. We must conclude that w is a palindrome. The proof is an induction on the number of steps in a derivation of w from P .

BASIS: If the derivation is one step, then it must use one of the three productions that do not have P in the body. That is, the derivation is $P \Rightarrow \epsilon$, $P \Rightarrow 0$, or $P \Rightarrow 1$. Since ϵ , 0 , and 1 are all palindromes, the basis is proven.

INDUCTION: Now, suppose that the derivation takes $n + 1$ steps, where $n \geq 1$, and the statement is true for all derivations of n steps. That is, if $P \xRightarrow{*} x$ in n steps, then x is a palindrome.

Consider an $(n + 1)$ -step derivation of w , which must be of the form

$$P \Rightarrow 0P0 \xRightarrow{*} 0x0 = w$$

or $P \Rightarrow 1P1 \xRightarrow{*} 1x1 = w$, since $n + 1$ steps is at least two steps, and the productions $P \rightarrow 0P0$ and $P \rightarrow 1P1$ are the only productions whose use allows additional steps of a derivation. Note that in either case, $P \xRightarrow{*} x$ in n steps.

By the inductive hypothesis, we know that x is a palindrome; that is, $x = x^R$. But if so, then $0x0$ and $1x1$ are also palindromes. For instance, $(0x0)^R = 0x^R0 = 0x0$. We conclude that w is a palindrome, which completes the proof.

□

1. $P \rightarrow \epsilon$
2. $P \rightarrow 0$
3. $P \rightarrow 1$
4. $P \rightarrow 0P0$
5. $P \rightarrow 1P1$

Figure 5.1: A context-free grammar for palindromes

Sentential Form

call these “sentential forms.” That is, if $G = (V, T, P, S)$ is a CFG, then any string α in $(V \cup T)^*$ such that $S \xRightarrow{*}_{lm} \alpha$ is a *sentential form*. If $S \xRightarrow{*}_{lm} \alpha$, then α is a *left-sentential form*, and if $S \xRightarrow{*}_{rm} \alpha$, then α is a *right-sentential form*.

Note that the language $L(G)$ is those sentential forms that are in T^* ; i.e., they consist solely of terminals.

$$E \xRightarrow{lm} E * E \xRightarrow{lm} I * E \xRightarrow{lm} a * E$$

Additionally, the derivation

$$E \xRightarrow{rm} E * E \xRightarrow{rm} E * (E) \xRightarrow{rm} E * (E + E)$$

shows that $E * (E + E)$ is a right-sentential form. $\square$

$$E \Rightarrow E * E \Rightarrow E * (E) \Rightarrow E * (E + E) \Rightarrow E * (I + E)$$

- Section 5.1 of Chapter 5